

## > Exercícios CAP. 2 (pág. 139) <

- ③ Porque precisam realizar operações profundas no hardware. Linguagens de alto nível não oferecem acesso direto a essas instruções de baixo nível exigidas em trocas de contexto.
- ⑤ A fração desperdiçada é a probabilidade de todos os programas aguardarem E/S simultaneamente.
- ①6 É possível. Para threads de Kernel, o SO utiliza isso naturalmente para preempção. Para as de usuário, isso ocorre se a biblioteca solicitar ao SO um sinal para forçar a troca.
- ①8 A maior vantagem é a velocidade extrema na troca de contexto, dispensando chamadas de sistema e transições para o modo núcleo.
- A maior desvantagem é o bloqueio do processo inteiro: se uma thread realizar uma operação de E/S bloqueante, todas as outras param junto.
- ②0 É essencial. Como a rotina não sabe antecipadamente o tamanho ou o tipo de dado a ser armazenado, ela usa um ponteiro de tamanho fixo. Assim, cada thread tem a flexibilidade de alocar dinamicamente a memória exata que precisa.
- ③1 Desabilitando as interrupções antes de testar, alterando o contador do semáforo ou bloqueando/acordando processos, reabilitando-os logo em seguida.
- ③3 Sim, faz sentido. Se esses dois processos cooperam dividindo o trabalho em fases, a barreira garante que nenhum deles inicie a próxima etapa antes que o outro termine a atual, mantendo a sincronização da tarefa.

41) Sim, no código-fonte é possível identificar. Em tempo de execução, o SO observa se o processo esgota todo o seu quantum frequentemente (CPU) ou se bloqueia voluntariamente aguardando E/S.

---

42) Se o quantum for muito curto, o tempo gasto nas trocas de contexto consome a CPU, derrubando a eficiência do sistema. Já um quantum muito longo, dilui o tempo, mas acaba com a interatividade.

---

53) O núcleo deve implementar o mecanismo da troca de contexto física entre as threads. A política deve ser delegada a um processo no espaço de usuário, que ditará as regras ao núcleo enviando parâmetros através de chamadas de sistema.

---